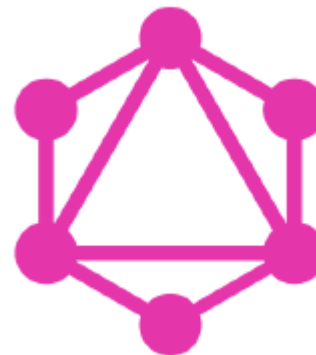


GraphQL

Desarrollo de Software en Sistemas Distribuidos

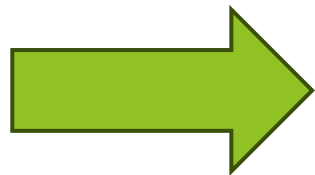
¿Qué es GraphQL?



- ▶ Lenguaje de consulta **de código abierto** para APIs
- ▶ Entorno de ejecución **del lado del servidor**
- ▶ Usa un **esquema fuertemente tipado** para definir entidades y relaciones
- ▶ Es usado por empresas como **Airbnb, Amazon Web Services, Paypal, IBM, Meta, etc.:** <https://graphql.org/users/>

GraphQL

- ▶ Creado en 2012 y liberado por Facebook en 2015
- ▶ Desde 2019, mantenido por la GraphQL Foundation, bajo la Linux Foundation



Lenguajes soportados

JavaScript	Go	PHP	Java / Kotlin	C# / .NET	Python
Swift / Objective-C	Rust	Ruby	Elixir	Scala	Flutter
Clojure	Haskell	C / C++	Elm	OCaml / Reason	Erlang
Julia	R	Groovy	Perl	D	

```
schema.gql x
schema.gql
27
28   type User {
29     id: ID!
30     username: String!
31     profile: Profile!
32     isActive: Boolean!
33     registeredAt: DateTime!
34     lastLogin: DateTime
35   }
36
```

Cinco pilares de GraphQL

- ▶ **Centrado en el producto** → diseñado para frontend.
- ▶ **Jerárquico** → consultas que reflejan la estructura de la UI.
- ▶ **Tipificación fuerte** → validación y resultados predecibles.
- ▶ **Respuesta definida por el cliente** → pide solo lo necesario.
- ▶ **Autodocumentación** → el esquema expone tipos y capacidades.

Casos de uso

- ▶ Grandes backends con múltiples servicios.
- ▶ Aplicaciones móviles.
- ▶ Apps con frontend complejo.
- ▶ Interfaces con actualizaciones en tiempo real.
- ▶ Proyectos con TypeScript o impulsados por IA.

Fundamentos de GraphQL

► Schema:

Es el contrato entre cliente y servidor.

Define tipos, queries, mutations y subscriptions.

► Tipos básicos:

Escalares: String, Int, Float, Boolean, ID.

Objetos: colecciones de campos.

Listas y tipos no nulos (String!, [User!]).

Tipos personalizados definidos por el servidor.

► Operaciones:

Query: lectura de datos.

Mutation: escritura o modificación de datos.

Subscription: eventos en tiempo real.

GraphQL - Operaciones

► Queries:

- Equivalente a los **GET** de REST.
- Sirven solo para **leer datos**.

► Ejemplo:

```
type User {  
  id: ID!  
  name: String!  
  email: String  
}  
  
type Query {  
  getUser(id: ID!): User  
}
```

► Uso:

```
query {  
  getUser(id: "1") {  
    id  
    name  
    email  
  }  
}
```


GraphQL - Operaciones

► Mutations:

- Equivalente a los **POST/PUT/PATCH/DELETE** de REST.
- Se usan para crear, actualizar o eliminar datos.

► Ejemplo:

```
type Mutation {  
  createUser(name: String!, email: String!): User  
}
```

► Uso:

```
mutation {  
  createUser(name: "Ana", email: "ana@example.com") {  
    id  
    name  
  }  
}
```

GraphQL - Operaciones

► Subscriptions:

- Permiten al cliente recibir actualizaciones automáticamente desde el servidor cuando ocurre un cambio en los datos.
- Se usan para notificaciones, chats en tiempo real, dashboards que se actualizan solos, etc.
- Funcionan mediante WebSockets (o protocolos que mantienen conexión abierta).

► Ejemplo:

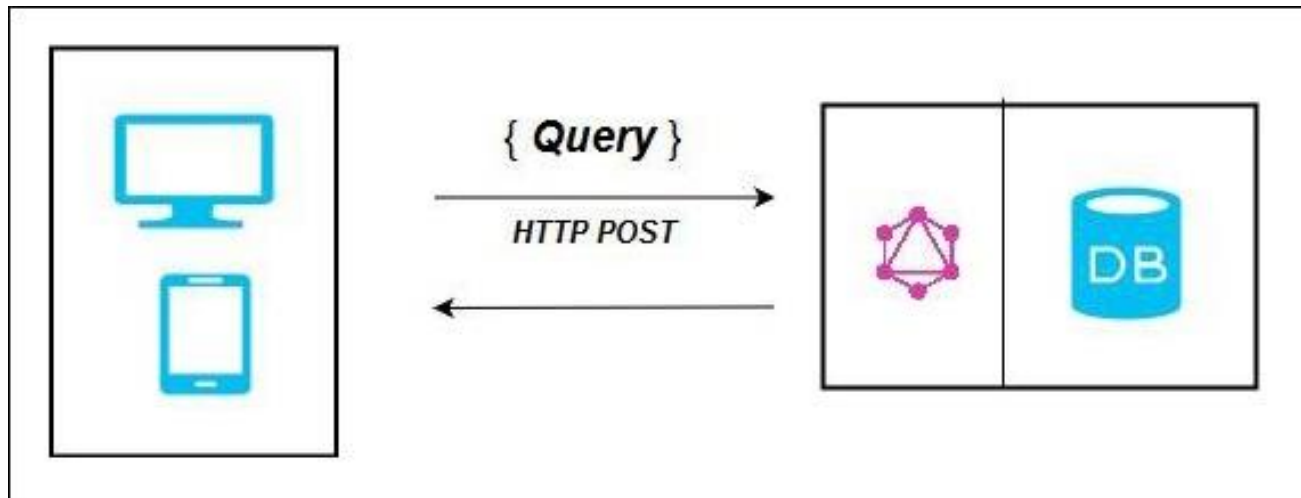
```
type Subscription {  
  newPostPublished: Post  
}
```

► Uso:

```
subscription {  
  newPostPublished {  
    title  
    author  
  }  
}
```

GraphQL - Arquitecturas

Servidor GraphQL con Base de Datos conectada



- Ideal para nuevos proyectos
- **Ventaja:** Simplicidad y control total sobre datos y consultas.

El cliente envía una query.

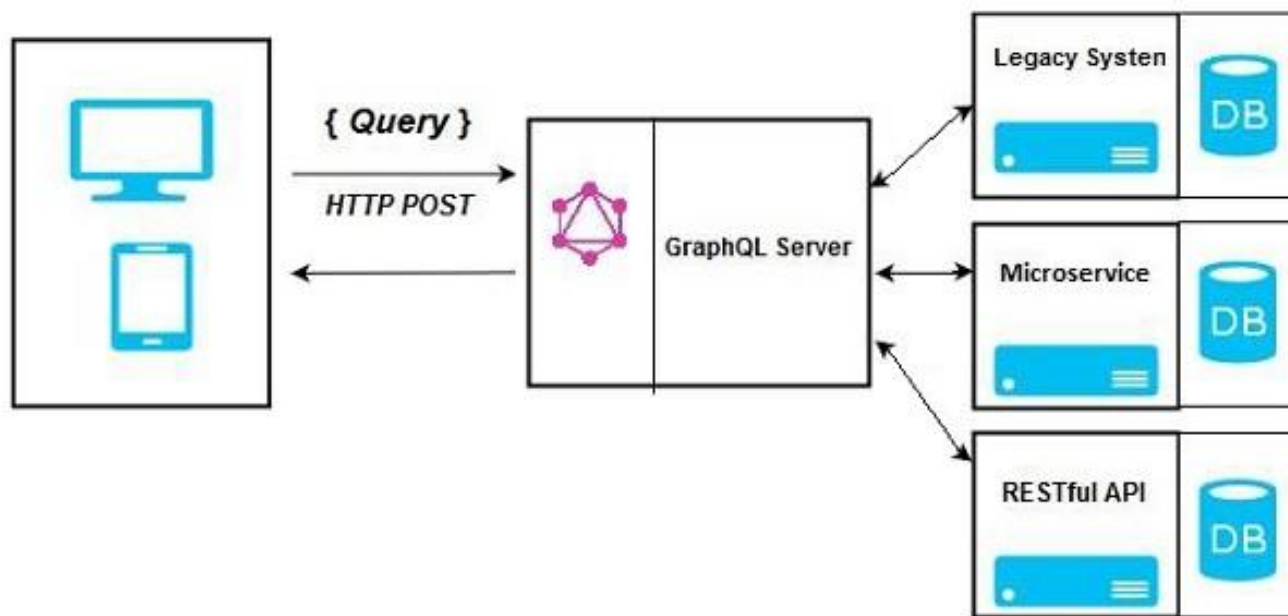
El servidor procesa la petición.

Se consulta la base de datos.

Se devuelve la respuesta en el formato estándar GraphQL.

GraphQL - Arquitecturas

Servidor GraphQL Integrando Sistemas Existentes

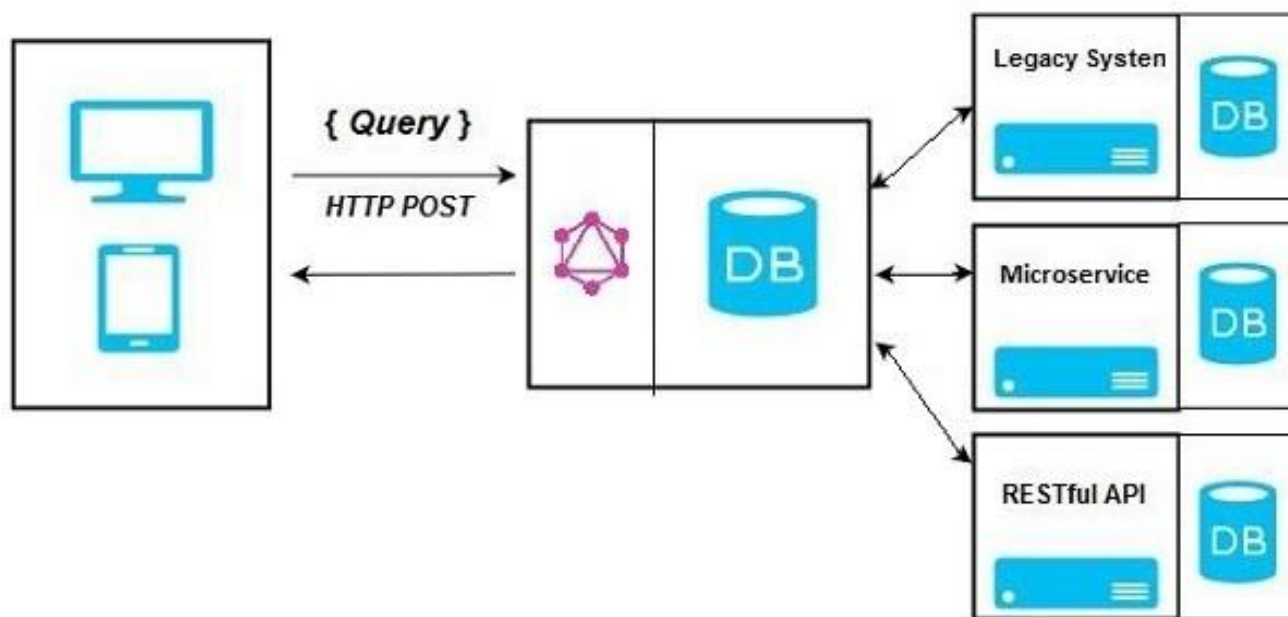


- Usado por **empresas con infraestructura heredada o múltiples APIs.**
- **Ventaja:** Se centraliza el acceso a datos heterogéneos bajo una sola API GraphQL.

El servidor GraphQL actúa como capa de unificación.

GraphQL - Arquitecturas

Enfoque Híbrido



- Combina los dos anteriores
- **Ventaja:** Flexibilidad para proyectos que necesitan datos propios y de terceros.

El servidor GraphQL resolverá cualquier solicitud recibida.

GraphQL - Ventajas

1. Eficiencia en la obtención de datos.

- ✓ Se reduce la cantidad de datos transferidos.
- ✓ Menos peticiones de red.

2. Desarrollo front-end más rápido.

- ✓ Los desarrolladores obtienen exactamente los datos que necesitan en una sola query.

3. APIs más estables.

- ✓ Los cambios en el back-end no rompen las consultas existentes.

4. Mejor documentación.

- ✓ El schema funciona como documentación viva y precisa de la API.

5. Soporte para la evolución de la API.

- ✓ Se pueden añadir nuevos campos sin afectar las consultas antiguas.

GraphQL - Implementación

- ▶ Ejemplo:

<https://github.com/JulianTomczak/graphqlApp>

- ▶ Front React:

<https://github.com/JulianTomczak/front-graphql>